

Βάσεις Δεδομένων

Αναφορά Εξαμηνιαίας Εργασίας – Ομάδα 21

Ελένη Νασοπούλου – 03121087

Ξενοφών Παπουγιαννάκης – 03121039

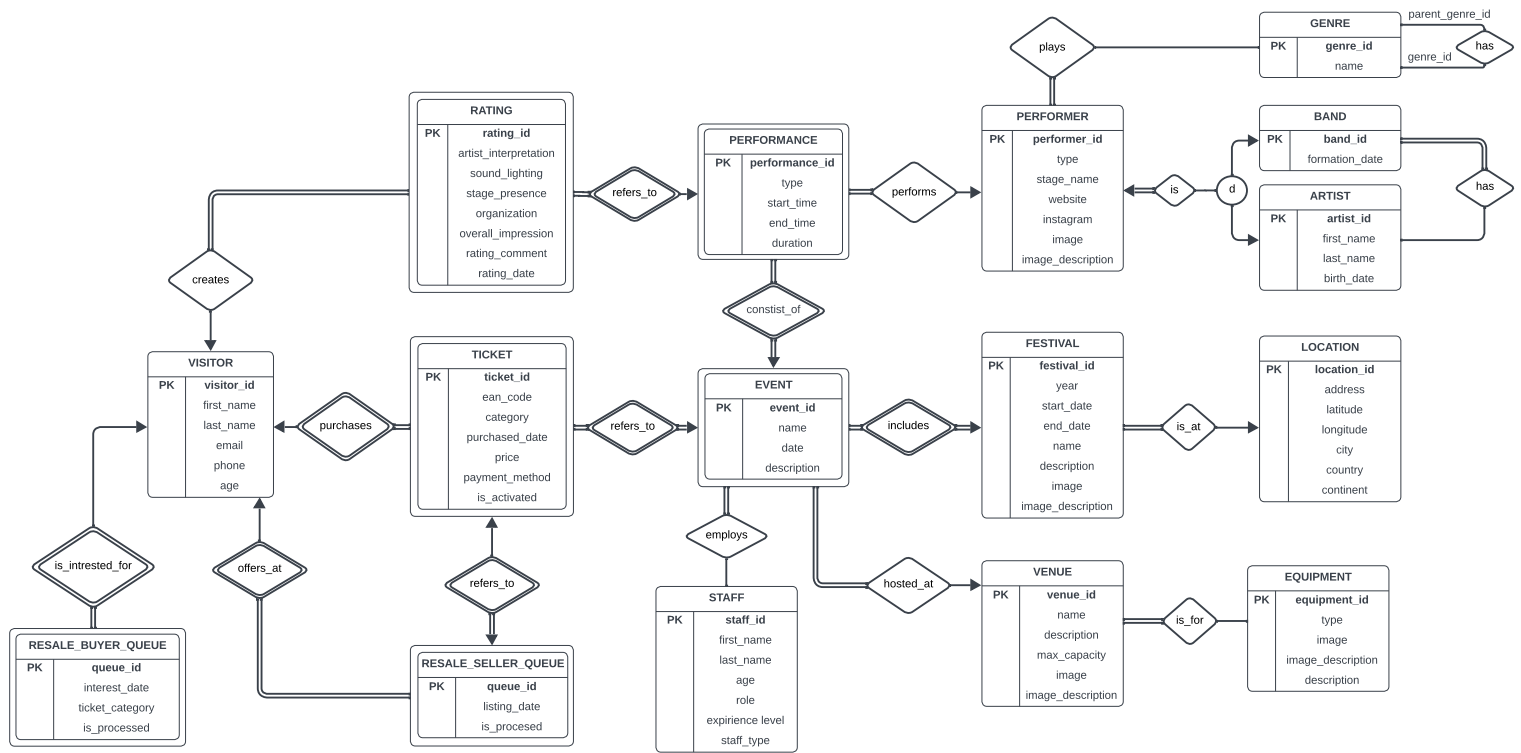
Παντελής Πετράκης – 03121060

Table of Contents

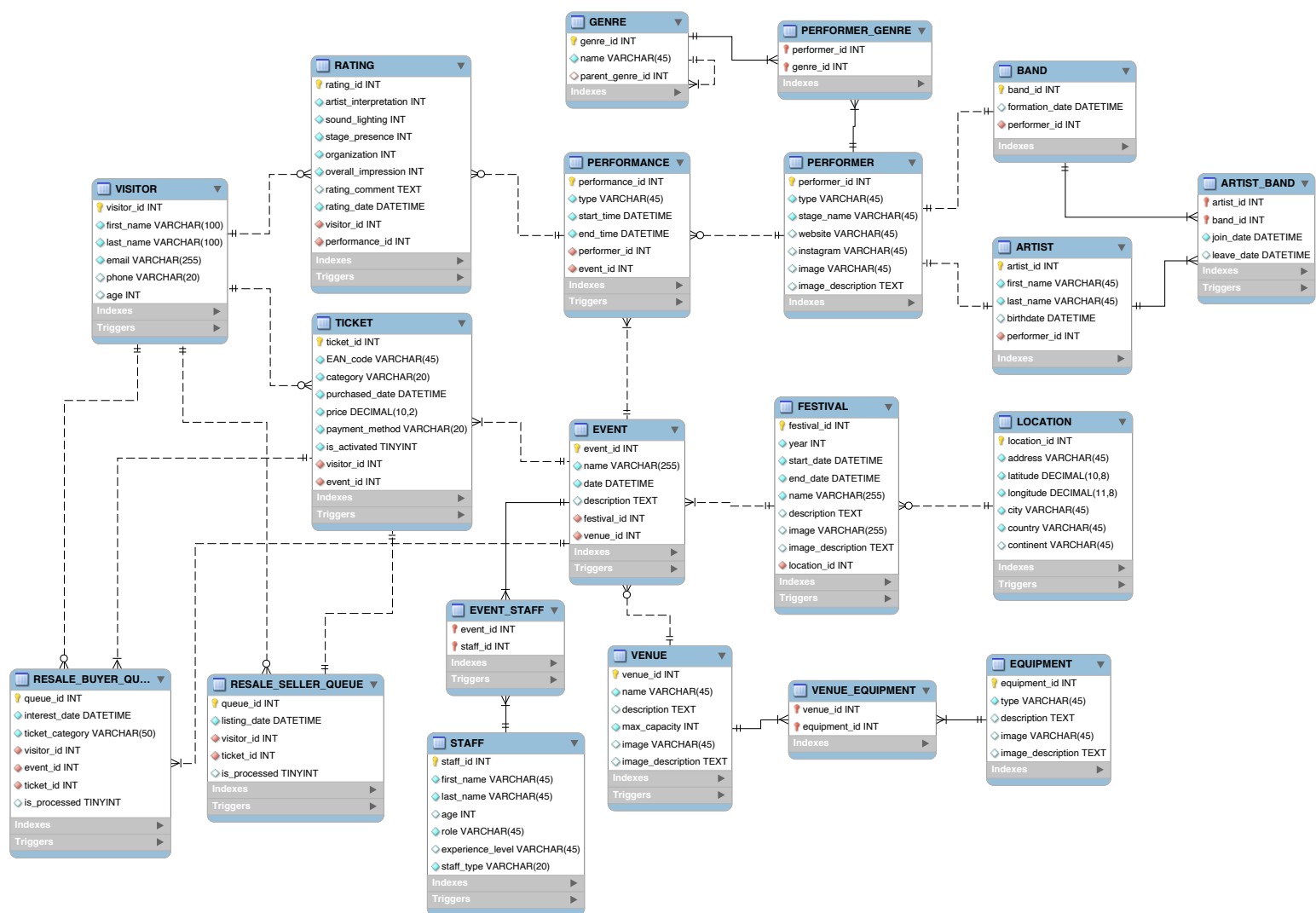
1. Diagrams	2
1.1 ER diagram	2
1.2 Relational diagram.....	3
2. Ευρετήρια (Indexes)	4
2.1 Μοναδικά Ευρετήρια (Unique Indexes).....	4
2.2 Ευρετήρια Ξένων Κλειδιών (Foreign Key Indexes)	5
2.3 Ειδικά Ευρετήρια για Βελτιστοποίηση Ερωτημάτων 4 και 6	7
2.4 Συνολική Αξιολόγηση των Ευρετηρίων	7
3. Triggers (Περιορισμοί)	8
4. Ανάλυση ερωτημάτων 4 και 6	10
4.1 Ερώτημα 4	10
4.2 Ερώτημα 6	15
5. Για την ανάπτυξη του project, χρησιμοποιήθηκαν:	21
6. Git Repository link.....	21

1. Diagrams

1.1 ER diagram



1.2 Relational diagram



2. Ευρετήρια (Indexes)

Τα ευρετήρια είναι κρίσιμα για τη βελτιστοποίηση της απόδοσης των ερωτημάτων στη βάση δεδομένων. Παρακάτω αναλύονται όλα τα ευρετήρια που έχουν οριστεί στη βάση δεδομένων μας και η χρησιμότητά τους για τα 15 ερωτήματα της εκφώνησης.

2.1 Μοναδικά Ευρετήρια (Unique Indexes)

1. address_UNIQUE στον πίνακα LOCATION

Διασφαλίζει ότι κάθε εγγραφή στον πίνακα LOCATION έχει μοναδική διεύθυνση.

- Ερώτημα 13: Επιταχύνει την αναζήτηση καλλιτεχνών που έχουν συμμετάσχει σε φεστιβάλ σε τουλάχιστον 3 διαφορετικές ηπείρους, καθώς επιτρέπει γρήγορη αναζήτηση τοποθεσιών με βάση τη διεύθυνση.

2. name_UNIQUE στον πίνακα FESTIVAL

Διασφαλίζει ότι κάθε εγγραφή στον πίνακα FESTIVAL έχει μοναδικό όνομα φεστιβάλ.

- Ερώτημα 1: Επιταχύνει την αναζήτηση εσόδων ανά φεστιβάλ και έτος.
- Ερώτημα 7: Διευκολύνει την εύρεση του φεστιβάλ με τον χαμηλότερο μέσο όρο εμπειρίας τεχνικού προσωπικού.

3. name_UNIQUE στον πίνακα VENUE

Διασφαλίζει ότι κάθε εγγραφή στον πίνακα VENUE έχει μοναδικό όνομα σκηνής.

- Ερώτημα 12: Επιταχύνει την αναζήτηση του προσωπικού που απαιτείται για κάθε ημέρα του φεστιβάλ, μέσω ταχείας εύρεσης της σκηνής βάσει ονόματος.

4. stage_name_UNIQUE στον πίνακα PERFORMER

Διασφαλίζει ότι κάθε εγγραφή στον πίνακα PERFORMER έχει μοναδικό καλλιτεχνικό όνομα.

- Ερώτημα 2: Επιταχύνει την αναζήτηση καλλιτεχνών που ανήκουν σε συγκεκριμένο μουσικό είδος.
- Ερώτημα 3: Διευκολύνει την αναζήτηση καλλιτεχνών που έχουν εμφανιστεί ως warm up περισσότερες από 2 φορές στο ίδιο φεστιβάλ.
- Ερώτημα 4: Επιταχύνει την ανάκτηση του μέσου όρου αξιολογήσεων για συγκεκριμένο καλλιτέχνη.
- Ερώτημα 5: Βοηθά στην εύρεση νέων καλλιτεχνών με τις περισσότερες συμμετοχές σε φεστιβάλ.
- Ερώτημα 11: Επιταχύνει την αναζήτηση καλλιτεχνών βάσει του αριθμού συμμετοχών τους.
- Ερώτημα 15: Διευκολύνει την αναζήτηση των top-5 επισκεπτών που έχουν δώσει τις υψηλότερες βαθμολογίες σε έναν καλλιτέχνη.

5. name_UNIQUE στον πίνακα GENRE

Διασφαλίζει ότι κάθε εγγραφή στον πίνακα GENRE έχει μοναδικό όνομα μουσικού είδους.

- Ερώτημα 2: Επιταχύνει την αναζήτηση καλλιτεχνών που ανήκουν σε συγκεκριμένο μουσικό είδος.
- Ερώτημα 10: Διευκολύνει την αναζήτηση των top-3 ζευγών μουσικών ειδών που είναι κοινά στους καλλιτέχνες.
- Ερώτημα 14: Επιταχύνει την αναζήτηση μουσικών ειδών με τον ίδιο αριθμό εμφανίσεων σε δύο συνεχόμενες χρονιές.

6. email_UNIQUE στον πίνακα VISITOR

Διασφαλίζει ότι κάθε εγγραφή στον πίνακα VISITOR έχει μοναδικό email επισκέπτη.

- Ερώτημα 6: Επιταχύνει την αναζήτηση παραστάσεων που έχει παρακολουθήσει ένας συγκεκριμένος επισκέπτης.
- Ερώτημα 9: Διευκολύνει την εύρεση επισκεπτών που έχουν παρακολουθήσει τον ίδιο αριθμό παραστάσεων.
- Ερώτημα 15: Επιταχύνει την αναζήτηση των top-5 επισκεπτών που έχουν δώσει την υψηλότερη βαθμολογία σε έναν καλλιτέχνη.

7. EAN_code_UNIQUE στον πίνακα TICKET

Διασφαλίζει ότι κάθε εγγραφή στον πίνακα TICKET έχει μοναδικό κωδικό EAN.

- Ερώτημα 1: Επιταχύνει την αναζήτηση εσόδων από την πώληση εισιτηρίων.
- Ερώτημα 9: Βοηθά στην αναζήτηση επισκεπτών που έχουν παρακολουθήσει τον ίδιο αριθμό παραστάσεων.

8. unique_visitor_performance στον πίνακα RATING

Διασφαλίζει ότι κάθε επισκέπτης μπορεί να αξιολογήσει μια συγκεκριμένη εμφάνιση μόνο μία φορά.

- Ερώτημα 4: Επιταχύνει την αναζήτηση του μέσου όρου αξιολογήσεων για συγκεκριμένο καλλιτέχνη.
- Ερώτημα 6: Βοηθά στην αναζήτηση του μέσου όρου αξιολόγησης ενός επισκέπτη ανά παράσταση.
- Ερώτημα 15: Επιταχύνει την αναζήτηση των top-5 επισκεπτών που έχουν δώσει την υψηλότερη βαθμολόγηση σε έναν καλλιτέχνη.

9. performer_id_UNIQUE στον πίνακα ARTIST

Διασφαλίζει ότι κάθε εγγραφή στον πίνακα ARTIST αντιστοιχεί σε έναν μοναδικό performer.

- Ερώτημα 5: Επιταχύνει την αναζήτηση νέων καλλιτεχνών (ηλικία < 30 ετών) με τις περισσότερες συμμετοχές σε φεστιβάλ.

10. performer_id_UNIQUE στον πίνακα BAND

Διασφαλίζει ότι κάθε εγγραφή στον πίνακα BAND αντιστοιχεί σε έναν μοναδικό performer.

- Ερώτημα 2: Επιταχύνει την αναζήτηση συγκροτημάτων που ανήκουν σε συγκεκριμένο μουσικό είδος.

11. ticket_id_UNIQUE στον πίνακα RESALE_SELLER_QUEUE

Διασφαλίζει ότι κάθε εισιτήριο μπορεί να είναι διαθέσιμο προς πώληση μόνο μία φορά.

- Βοηθά στη διαχείριση της ουράς μεταπώλησης εισιτηρίων, αποτρέποντας διπλές καταχωρήσεις του ίδιου εισιτηρίου.

2.2 Ευρετήρια Ξένων Κλειδιών (Foreign Key Indexes)

12. location_id_idx στον πίνακα FESTIVAL

Διασφαλίζει γρήγορη αναζήτηση φεστιβάλ με βάση την τοποθεσία.

- Ερώτημα 13: Βοηθά στην αναζήτηση καλλιτεχνών που έχουν συμμετάσχει σε φεστιβάλ σε τουλάχιστον 3 διαφορετικές ηπείρους.

13. festival_id_idx στον πίνακα EVENT

Επιταχύνει τις αναζητήσεις παραστάσεων με βάση το φεστιβάλ.

- Ερώτημα 1: Βοηθά στην αναζήτηση εσόδων ανά φεστιβάλ και έτος.
- Ερώτημα 2: Επιταχύνει την αναζήτηση καλλιτεχνών που συμμετείχαν σε εκδηλώσεις του φεστιβάλ για συγκεκριμένο έτος.
- Ερώτημα 3: Βοηθά στην αναζήτηση καλλιτεχνών που έχουν εμφανιστεί ως warm up περισσότερες από 2 φορές στο ίδιο φεστιβάλ.
- Ερώτημα 7: Επιταχύνει την αναζήτηση του φεστιβάλ με τον χαμηλότερο μέσο όρο εμπειρίας τεχνικού προσωπικού.
- Ερώτημα 12: Βοηθά στην αναζήτηση του προσωπικού που απαιτείται για κάθε ημέρα του φεστιβάλ.

14. venue_id_idx στον πίνακα EVENT

Επιταχύνει τις αναζητήσεις παραστάσεων με βάση τη σκηνή.

- Ερώτημα 12: Βοηθά στην αναζήτηση του προσωπικού που απαιτείται για κάθε ημέρα του φεστιβάλ, καθώς επιτρέπει γρήγορη αναζήτηση παραστάσεων με βάση τη σκηνή.

15. performer_id_idx στον πίνακα PERFORMANCE

Επιτρέπει γρήγορη αναζήτηση εμφανίσεων με βάση τον καλλιτέχνη.

- Ερώτημα 3: Βοηθά στην αναζήτηση warm up εμφανίσεων στο ίδιο φεστιβάλ.
- Ερώτημα 4: Επιταχύνει την ανάκτηση του μέσου όρου αξιολογήσεων για συγκεκριμένο καλλιτέχνη.
- Ερώτημα 5: Βοηθά στον εντοπισμό νέων καλλιτεχνών με πολλές συμμετοχές.
- Ερώτημα 11: Επιταχύνει την αναζήτηση καλλιτεχνών βάσει του αριθμού συμμετοχών.

- Ερώτημα 13: Βοηθά στην εύρεση καλλιτεχνών που συμμετείχαν σε 3+ ηπείρους.

16. event_id_idx στον πίνακα PERFORMANCE

Διευκολύνει την αναζήτηση εμφανίσεων με βάση την παράσταση.

- Ερώτημα 3: Βοηθά στην αναζήτηση καλλιτεχνών που έχουν εμφανιστεί ως warm up περισσότερες από 2 φορές στο ίδιο φεστιβάλ.
- Ερώτημα 6: Επιταχύνει την αναζήτηση παραστάσεων που έχει παρακολουθήσει ένας επισκέπτης.
- Ερώτημα 14: Διευκολύνει την ανάλυση ειδών με τον ίδιο αριθμό εμφανίσεων σε δύο συνεχόμενες χρονιές.

17. visitor_id_idx στους πίνακες TICKET, RATING, RESALE_BUYER_QUEUE, RESALE_SELLER_QUEUE

Επιταχύνει τις αναζητήσεις που σχετίζονται με τον επισκέπτη.

- Ερώτημα 6: Βοηθά στην αναζήτηση παραστάσεων και μέσου όρου αξιολόγησης επισκέπτη.
- Ερώτημα 9: Επιτρέπει την εύρεση επισκεπτών με τον ίδιο αριθμό παραστάσεων.
- Ερώτημα 15: Επιταχύνει την ανάλυση top-5 επισκεπτών με υψηλότερη βαθμολόγηση σε έναν καλλιτέχνη.

18. event_id_idx στους πίνακες TICKET, RESALE_BUYER_QUEUE

Επιτρέπει γρήγορη αναζήτηση εισιτηρίων με βάση την παράσταση.

- Ερώτημα 1: Βοηθά στην αναζήτηση εσόδων από την πώληση εισιτηρίων ανά παράσταση.
- Ερώτημα 9: Διευκολύνει την εύρεση επισκεπτών με ίδιο αριθμό παραστάσεων.

19. performance_id_idx στον πίνακα RATING

Επιταχύνει τις αναζητήσεις αξιολογήσεων με βάση την εμφάνιση.

- Ερώτημα 4: Επιταχύνει την ανάκτηση μέσου όρου αξιολογήσεων για συγκεκριμένο καλλιτέχνη.
- Ερώτημα 6: Διευκολύνει τον υπολογισμό μέσου όρου αξιολόγησης επισκέπτη ανά παράσταση.
- Ερώτημα 15: Επιτρέπει εύκολη αναζήτηση top-5 επισκεπτών.

20. ticket_id_idx στους RESALE_BUYER_QUEUE, RESALE_SELLER_QUEUE

Επιτρέπει αναζήτηση εγγραφών μεταπώλησης με βάση το εισιτήριο.

- Βοηθά στη σωστή διαχείριση της ουράς μεταπώλησης, επιτρέποντας γρήγορη αναζήτηση εγγραφών με βάση το εισιτήριο.

21. staff_id_idx στον πίνακα EVENT_STAFF

Διευκολύνει αναζητήσεις με βάση το μέλος προσωπικού.

- Ερώτημα 7: Βοηθά στην αναζήτηση του φεστιβάλ με τον χαμηλότερο μέσο όρο εμπειρίας τεχνικού προσωπικού.
- Ερώτημα 8: Επιταχύνει την αναζήτηση του προσωπικού υποστήριξης που δεν έχει προγραμματισμένη εργασία σε συγκεκριμένη ημερομηνία.
- Ερώτημα 12: Ανάθεση προσωπικού που απαιτείται ανά ημέρα φεστιβάλ.

22. equipment_id_idx στον πίνακα VENUE_EQUIPMENT

Επιταχύνει αναζητήσεις εξοπλισμού σκηνών.

- Βοηθά στη διαχείριση του εξοπλισμού των σκηνών, επιτρέποντας γρήγορη αναζήτηση σκηνών που διαθέτουν συγκεκριμένο εξοπλισμό.

23. genre_id_idx στον πίνακα PERFORMER_GENRE

Επιτρέπει γρήγορη αναζήτηση καλλιτεχνών βάσει μουσικού είδους.

- Ερώτημα 2: Επιταχύνει την αναζήτηση καλλιτεχνών που ανήκουν σε συγκεκριμένο μουσικό είδος.
- Ερώτημα 10: Βοηθά στην αναζήτηση των top-3 ζευγών μουσικών ειδών που είναι κοινά στους καλλιτέχνες.
- Ερώτημα 14: Επιταχύνει την αναζήτηση μουσικών ειδών με τον ίδιο αριθμό εμφανίσεων σε δύο συνεχόμενες χρονιές.

24. band_id_idx στον πίνακα ARTIST_BAND

Διευκολύνει την αναζήτηση μελών συγκροτημάτων.

- Ερώτημα 2: Βοηθά στην αναζήτηση καλλιτεχνών που ανήκουν σε συγκεκριμένο μουσικό είδος, συμπεριλαμβανομένων των μελών συγκροτημάτων.

25. parent_genre_id_idx στον πίνακα GENRE

Επιταχύνει τις αναζητήσεις ειδών με βάση το γονικό είδος.

- Ερώτημα 2: Βοηθά στην αναζήτηση καλλιτεχνών που ανήκουν σε συγκεκριμένο μουσικό είδος, συμπεριλαμβανομένων των υποειδών.
- Ερώτημα 10: Επιταχύνει την αναζήτηση των top-3 ζευγών μουσικών ειδών που είναι κοινά στους καλλιτέχνες.
- Ερώτημα 14: Βοηθά στην αναζήτηση μουσικών ειδών με τον ίδιο αριθμό εμφανίσεων σε δύο συνεχόμενες χρονιές.

2.3 Ειδικά Ευρετήρια για Βελτιστοποίηση Ερωτημάτων 4 και 6

26. idx_performer_stage_name στον πίνακα PERFORMER

Δημιουργήθηκε ειδικά για το ερώτημα 4, επιταχύνοντας αναζητήσεις καλλιτεχνών με βάση το καλλιτεχνικό όνομα.

- Ερώτημα 4: Επιταχύνει την αναζήτηση του μέσου όρου αξιολογήσεων για συγκεκριμένο καλλιτέχνη.

27. idx_performance_performer_id στον πίνακα PERFORMANCE

Βελτιστοποιεί τις συνδέσεις (joins) μεταξύ των πινάκων PERFORMANCE και PERFORMER.

- Ερώτημα 4: Επιταχύνει την αναζήτηση του μέσου όρου αξιολογήσεων για συγκεκριμένο καλλιτέχνη.

28. idx_rating_performance_id στον πίνακα RATING

Βελτιστοποιεί τις συνδέσεις μεταξύ των πινάκων RATING και PERFORMANCE.

- Ερώτημα 4: Επιταχύνει την αναζήτηση του μέσου όρου αξιολογήσεων για συγκεκριμένο καλλιτέχνη.
- Ερώτημα 6: Βοηθά στην αναζήτηση του μέσου όρου αξιολόγησης ενός επισκέπτη ανά παράσταση.

29. idx_visitor_email στον πίνακα VISITOR

Δημιουργήθηκε ειδικά για το ερώτημα 6, επιταχύνοντας αναζητήσεις επισκεπτών με βάση το email.

- Ερώτημα 6: Επιταχύνει την αναζήτηση παραστάσεων που έχει παρακολουθήσει ένας συγκεκριμένος επισκέπτης και του μέσου όρου αξιολόγησής του.

30. idx_rating_visitor_id στον πίνακα RATING

Βελτιστοποιεί τις συνδέσεις μεταξύ των πινάκων RATING και VISITOR.

- Ερώτημα 6: Επιταχύνει την αναζήτηση του μέσου όρου αξιολόγησης ενός επισκέπτη ανά παράσταση.

2.4 Συνολική Αξιολόγηση των Ευρετηρίων

Τα ευρετήρια που έχουν οριστεί στη βάση δεδομένων του φεστιβάλ μουσικής είναι προσεκτικά σχεδιασμένα για να βελτιστοποιήσουν την απόδοση των 15 ερωτημάτων της εκφώνησης.

Συγκεκριμένα:

1. Μοναδικά Ευρετήρια: Εξασφαλίζουν την ακεραιότητα των δεδομένων και επιταχύνουν αναζητήσεις με βάση μοναδικά χαρακτηριστικά όπως ονόματα, emails και κωδικούς.
2. Ευρετήρια Ξένων Κλειδιών: Βελτιστοποιούν τις συνδέσεις (joins) μεταξύ των πινάκων, επιταχύνοντας ερωτήματα που απαιτούν δεδομένα από πολλαπλούς πίνακες.
3. Ειδικά Ευρετήρια: Δημιουργήθηκαν για συγκεκριμένα ερωτήματα (τα 4 και 6) για να βελτιστοποιήσουν την απόδοσή τους.

Η χρήση των κατάλληλων ευρετηρίων είναι ιδιαίτερα σημαντική για τα ερωτήματα που περιλαμβάνουν πολλαπλά joins και συναθροιστικές λειτουργίες (όπως τα ερωτήματα 4, 6, 10 και 15), καθώς και για

ερωτήματα που απαιτούν ταξινόμηση και ομαδοποίηση μεγάλου όγκου δεδομένων (όπως τα ερωτήματα 1, 9 και 11).

Συνολικά, τα ευρετήρια που έχουν οριστεί καλύπτουν όλες τις απαιτήσεις των ερωτημάτων της εκφώνησης, εξασφαλίζοντας βέλτιστη απόδοση και αποτελεσματική διαχείριση των δεδομένων του φεστιβάλ μουσικής.

3. Triggers (Περιορισμοί)

Οι triggers που έχουν οριστεί στη βάση δεδομένων μας εξασφαλίζουν την ακεραιότητα των δεδομένων και την τήρηση των επιχειρησιακών κανόνων. Παρακάτω αναλύονται όλοι οι triggers.

1. Triggers για τον πίνακα LOCATION

- `trg_location_validate_coordinates_before_insert`:
Ελέγχει ότι οι συντεταγμένες γεωγραφικού πλάτους και μήκους είναι έγκυρες (-90 έως 90 για το πλάτος και -180 έως 180 για το μήκος) κατά την εισαγωγή.
- `trg_location_validate_coordinates_before_update`:
Ελέγχει ότι οι συντεταγμένες γεωγραφικού πλάτους και μήκους είναι έγκυρες (-90 έως 90 για το πλάτος και -180 έως 180 για το μήκος) κατά την ενημέρωση.

2. Triggers για τον πίνακα FESTIVAL

- `trg_festival_validate_dates_before_insert`:
Ελέγχει ότι η ημερομηνία λήξης είναι μεταγενέστερη της ημερομηνίας έναρξης και ότι και οι δύο ημερομηνίες ανήκουν στο έτος του φεστιβάλ.
- `trg_festival_prevent_delete`:
Αποτρέπει τη διαγραφή φεστιβάλ, σύμφωνα με τον κανόνα ότι τα φεστιβάλ δεν μπορούν να ακυρωθούν.

3. Triggers για τον πίνακα VENUE

- `trg_venue_validate_capacity_before_insert`:
Ελέγχει ότι η μέγιστη χωρητικότητα της σκηνής είναι θετικός αριθμός κατά την εισαγωγή.
- `trg_venue_validate_capacity_before_update`:
Ελέγχει ότι η μέγιστη χωρητικότητα της σκηνής είναι θετικός αριθμός κατά την ενημέρωση.

4. Triggers για τον πίνακα EVENT

- `trg_event_validate_before_insert`:
Ελέγχει ότι η ημερομηνία της παράστασης είναι εντός των ημερομηνιών του φεστιβάλ και ότι δεν υπάρχει άλλη παράσταση στην ίδια σκηνή την ίδια ημέρα.
- `trg_event_prevent_deletion`:
Αποτρέπει τη διαγραφή παραστάσεων, σύμφωνα με τον κανόνα ότι οι παραστάσεις δεν μπορούν να ακυρωθούν.

5. Triggers για τον πίνακα STAFF

- `trg_staff_validate_before_insert`:
Ελέγχει ότι το επίπεδο εμπειρίας και ο τύπος του προσωπικού είναι εντός των αποδεκτών τιμών κατά την εισαγωγή.
- `trg_staff_validate_before_update`:
Ελέγχει ότι το επίπεδο εμπειρίας και ο τύπος του προσωπικού είναι εντός των αποδεκτών τιμών κατά την ενημέρωση.

6. Triggers για τον πίνακα PERFORMANCE

- `trg_performance_validate_schedule_before_insert`:
Ελέγχει ότι η ώρα λήξης είναι μεταγενέστερη της ώρας έναρξης, ότι η εμφάνιση πραγματοποιείται την ίδια ημέρα με την παράσταση, ότι δεν υπάρχουν επικαλυπτόμενες εμφανίσεις του ίδιου καλλιτέχνη και ότι η διάρκεια δεν υπερβαίνει τις 3 ώρες.

- `trg_performance_enforce_intervals_before_insert`:
Ελέγχει ότι τα διαλείμματα μεταξύ διαδοχικών εμφανίσεων είναι τουλάχιστον 5 λεπτά και το πολύ 30 λεπτά.
- `trg_performance_limit_consecutive_years_before_insert`:
Ελέγχει ότι ένας καλλιτέχνης δεν συμμετέχει για περισσότερα από 3 συνεχόμενα έτη.

7. Triggers για τον πίνακα VISITOR

- `trg_visitor_validate_age_before_insert`:
Ελέγχει ότι η ηλικία του επισκέπτη είναι μεταξύ 0 και 100 ετών κατά την εισαγωγή.
- `trg_visitor_validate_age_before_update`:
Ελέγχει ότι η ηλικία του επισκέπτη είναι μεταξύ 0 και 100 ετών κατά την ενημέρωση.

8. Triggers για τον πίνακα TICKET

- `trg_ticket_validate_before_insert`:
Ελέγχει ότι το εισιτήριο αγοράζεται πριν την ημερομηνία της παράστασης, ότι η παράσταση δεν είναι sold out, ότι ο επισκέπτης δεν έχει ήδη εισιτήριο για την ίδια παράσταση, ότι τα VIP εισιτήρια δεν υπερβαίνουν το 10% της χωρητικότητας και ότι η τιμή είναι θετική.
- `trg_ticket_validate_before_update`:
Ελέγχει ότι η τιμή είναι θετική, ότι το πεδίο `is_activated` είναι 0 ή 1, ότι η κατηγορία και ο τρόπος πληρωμής είναι εντός των αποδεκτών τιμών, επιτρέπει την ενεργοποίηση εισιτηρίων αλλά αποτρέπει την απενεργοποίηση ήδη ενεργοποιημένων εισιτηρίων.

9. Triggers για τον πίνακα RATING

- `trg_rating_validate_before_insert`:
Ελέγχει ότι η αξιολόγηση υποβάλλεται μετά το τέλος της εμφάνισης, ότι ο επισκέπτης έχει εισιτήριο για την παράσταση, ότι κάθε βαθμολογία είναι μεταξύ 1 και 5 και ότι δεν υπάρχει διπλή αξιολόγηση για την ίδια εμφάνιση από τον ίδιο επισκέπτη κατά την εισαγωγή.
- `trg_rating_validate_values_before_update`:
Ελέγχει ότι η αξιολόγηση υποβάλλεται μετά το τέλος της εμφάνισης, ότι ο επισκέπτης έχει εισιτήριο για την παράσταση, ότι κάθε βαθμολογία είναι μεταξύ 1 και 5 και ότι δεν υπάρχει διπλή αξιολόγηση για την ίδια εμφάνιση από τον ίδιο επισκέπτη κατά την ενημέρωση.

10. Triggers για πίνακες RESALE_BUYER_QUEUE και RESALE_SELLER_QUEUE

- `trg_resale_validate_buyer_before_insert`:
Ελέγχει ότι ο αγοραστής εκδηλώνει ενδιαφέρον είτε για συγκεκριμένη παράσταση και κατηγορία εισιτηρίου, είτε για συγκεκριμένο εισιτήριο, και ότι το εισιτήριο είναι διαθέσιμο προς πώληση.
- `trg_resale_validate_seller_before_insert`:
Ελέγχει ότι ο πωλητής είναι ο ιδιοκτήτης του εισιτηρίου, ότι το εισιτήριο δεν έχει ενεργοποιηθεί και ότι δεν είναι ήδη καταχωρημένο προς πώληση.

11. Triggers για τον πίνακα EVENT_STAFF

- `trg_event_staff_requirements_before_insert`:
Ελέγχει ότι ο τύπος του προσωπικού είναι έγκυρος και υπολογίζει τις απαιτήσεις προσωπικού με βάση τη χωρητικότητα της σκηνής.
- `trg_event_staff_requirements_before_delete`:
Αποτρέπει τη διαγραφή προσωπικού αν αυτό παραβιάζει τις ελάχιστες απαιτήσεις προσωπικού (5% για προσωπικό ασφαλείας και 2% για βοηθητικό προσωπικό).

12. Triggers για τον πίνακα ARTIST_BAND

- `trg_artist_band_validate_before_insert`:
Ελέγχει ότι η ημερομηνία αποχώρησης (αν υπάρχει) είναι μεταγενέστερη της ημερομηνίας ένταξης και ότι ο καλλιτέχνης δεν είναι ήδη μέλος του συγκροτήματος.

4. Ανάλυση ερωτημάτων 4 και 6

4.1 Ερώτημα 4

Το ερώτημα 4 αφορά τον υπολογισμό του μέσου όρου αξιολογήσεων για έναν συγκεκριμένο καλλιτέχνη. Στην παρούσα ανάλυση, θα εξετάσουμε διαφορετικές στρατηγικές βελτιστοποίησης για την εκτέλεση αυτού του ερωτήματος, με στόχο την επίτευξη της βέλτιστης απόδοσης.

4.1.1 Αρχικό Ερώτημα

Το αρχικό SQL ερώτημα που χρησιμοποιήθηκε για τον υπολογισμό του μέσου όρου αξιολογήσεων είναι το εξής:

```
SELECT
    p.stage_name AS 'Performer',
    ROUND(AVG(r.artist_interpretation), 2) AS 'Avg Artist Interpretation',
    ROUND(AVG(r.overall_impression), 2) AS 'Avg Overall Impression'
FROM
    RATING r
JOIN
    PERFORMANCE perf ON r.performance_id = perf.performance_id
JOIN
    PERFORMER p ON perf.performer_id = p.performer_id
WHERE
    p.stage_name = @artist_name
GROUP BY
    p.performer_id;
```

Πλάνο Εκτέλεσης του Αρχικού Ερωτήματος (Default Join Strategy):

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	p	const	PRIMARY,stage_name_UNIQUE,idx_performer_stage_name	stage_name_UNIQUE	137	const	1	Using index
1	SIMPLE	perf	ref	PRIMARY,performer_id_idx,idx_performance_performer_id	performer_id_idx	4	const	2	Using index
1	SIMPLE	r	ref	performance_id_idx,idx_rating_performance_id	performance_id_idx	4	festival.perf.performance_id	1	

Από το πλάνο εκτέλεσης, παρατηρούμε ότι ο βελτιστοποιητής επιλέγει:

- Για τον πίνακα PERFORMER (p): Χρησιμοποιεί το ευρετήριο stage_name_UNIQUE για γρήγορη αναζήτηση με βάση το όνομα του καλλιτέχνη.
- Για τον πίνακα PERFORMANCE (perf): Χρησιμοποιεί το ευρετήριο performer_id_idx για να βρει τις εμφανίσεις του συγκεκριμένου καλλιτέχνη.
- Για τον πίνακα RATING (r): Χρησιμοποιεί το ευρετήριο performance_id_idx για να βρει τις αξιολογήσεις για τις συγκεκριμένες εμφανίσεις.

Ο χρόνος εκτέλεσης αυτού του ερωτήματος είναι 0.00041400 δευτερόλεπτα.

4.2.1 Ανάλυση Διαφορετικών Στρατηγικών Join

ι. Στρατηγική Nested Loop Join

Η στρατηγική Nested Loop Join υλοποιήθηκε με το ακόλουθο SQL ερώτημα:

```
SELECT STRAIGHT_JOIN
  p.stage_name AS 'Performer',
  ROUND(AVG(r.artist_interpretation), 2) AS 'Avg Artist Interpretation',
  ROUND(AVG(r.overall_impression), 2) AS 'Avg Overall Impression'
FROM
  PERFORMER p
JOIN
  PERFORMANCE perf USE INDEX (PRIMARY) ON p.performer_id = perf.performer_id
JOIN
  RATING r USE INDEX (PRIMARY) ON perf.performance_id = r.performance_id
WHERE
  p.stage_name = @artist_name
GROUP BY
  p.performer_id;
```

Το ερώτημα χρησιμοποιεί:

- STRAIGHT_JOIN: Αναγκάζει τον βελτιστοποιητή να εκτελέσει τα joins με τη σειρά που καθορίζονται στο ερώτημα.
- USE INDEX (PRIMARY): Προτείνει στον βελτιστοποιητή να χρησιμοποιήσει το πρωτεύον ευρετήριο του πίνακα PERFORMANCE.

Πλάνο Εκτέλεσης Nested Loop Join:

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	p	const	PRIMARY,stage_name_UNIQUE,idx_performer_stage_name	stage_name_UNIQUE	137	const	1	Using index
1	SIMPLE	perf	ALL	PRIMARY	NULL	NULL	NULL	175	Using where
1	SIMPLE	r	ALL	NULL	NULL	NULL	NULL	80	Using where; Using join buffer (flat, BNL join)

Από το πλάνο εκτέλεσης, παρατηρούμε ότι:

- Για τον πίνακα PERFORMER (p): Χρησιμοποιεί το ευρετήριο stage_name_UNIQUE.
- Για τον πίνακα PERFORMANCE (perf): Δεν χρησιμοποιεί κανένα ευρετήριο (type: ALL), παρά την οδηγία USE INDEX (PRIMARY).
- Για τον πίνακα RATING (r): Δεν χρησιμοποιεί κανένα ευρετήριο (type: ALL) και χρησιμοποιεί join buffer.

Αποτελέσματα Profiling για Nested Loop Join:

Status	Duration
Starting	0.000048
checking permissions	0.000004
Opening tables	0.000010
After opening tables	0.000003
System lock	0.000003
table lock	0.000004
init	0.000018
Optimizing	0.000018
Statistics	0.000031
Preparing	0.000003
Unlocking tables	0.000003
Preparing	0.000020
Executing	0.000002
Sending data	0.000151
End of update loop	0.000005
Query end	0.000003
Commit	0.000004
Query end	0.000002
closing tables	0.000003
Unlocking tables	0.000003
closing tables	0.000003
Query end	0.000004
Starting cleanup	0.000002
Freeing items	0.000006
Updating status	0.000021
Reset for next command	0.000003

Ο χρόνος εκτέλεσης αυτής της στρατηγικής είναι 0.00037700 δευτερόλεπτα, που είναι περίπου 9% ταχύτερος από το αρχικό ερώτημα.

Η στρατηγική Nested Loop Join λειτουργεί καλύτερα όταν ένας από τους πίνακες είναι μικρός και ο άλλος έχει καλό ευρετήριο στη στήλη join. Στην περίπτωση μας, ο πίνακας PERFORMER είναι μικρός (μόνο ένας καλλιτέχνης μετά το φιλτράρισμα), αλλά η έλλειψη χρήσης ευρετηρίων για τους πίνακες PERFORMANCE και RATING περιορίζει την απόδοση.

ii. Στρατηγική Hash Join (BNL)

Η στρατηγική Hash Join υλοποιήθηκε με το ακόλουθο SQL ερώτημα:

```
SELECT
  p.stage_name AS 'Performer',
  ROUND(AVG(r.artist_interpretation), 2) AS 'Avg Artist Interpretation',
  ROUND(AVG(r.overall_impression), 2) AS 'Avg Overall Impression'
FROM
  PERFORMER p
JOIN
  PERFORMANCE perf IGNORE INDEX (PRIMARY, idx_performance_performer_id) ON p.performer_id = perf.performer_id
JOIN
  RATING r IGNORE INDEX (PRIMARY, idx_rating_performance_id) ON perf.performance_id = r.performance_id
WHERE
  p.stage_name = @artist_name
GROUP BY
  p.performer_id;
```

Το ερώτημα χρησιμοποιεί:

- IGNORE INDEX (PRIMARY, idx_performance_performer_id): Αποτρέπει τον βελτιστοποιητή από το να χρησιμοποιήσει τα συγκεκριμένα ευρετήρια, αναγκάζοντάς τον να χρησιμοποιήσει μια στρατηγική Hash Join.

Πλάνο Εκτέλεσης Hash Join:

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	p	const	PRIMARY,stage_name_UNIQUE,idx_performer_stage_name	stage_name_UNIQUE	137	const	1	Using index
1	SIMPLE	perf	ref	performer_id_idx	performer_id_idx	4	const	2	Using index
1	SIMPLE	r	ref	performance_id_idx	performance_id_idx	4	festival.perf.performance_id	1	

Από το πλάνο εκτέλεσης, παρατηρούμε ότι:

- Για τον πίνακα PERFORMER (p): Χρησιμοποιεί το ευρετήριο stage_name_UNIQUE.
- Για τον πίνακα PERFORMANCE (perf): Χρησιμοποιεί το ευρετήριο performer_id_idx, παρά την οδηγία IGNORE INDEX.
- Για τον πίνακα RATING (r): Χρησιμοποιεί το ευρετήριο performance_id_idx.

Αποτελέσματα Profiling για Hash Join:

Status	Duration
Starting	0.000047
checking permissions	0.000003
Opening tables	0.000009
After opening tables	0.000002
System lock	0.000004
table lock	0.000003
init	0.000017
Optimizing	0.000015
Statistics	0.000047
Preparing	0.000004
Unlocking tables	0.000002
Preparing	0.000018
Executing	0.000003
Sending data	0.000029
End of update loop	0.000004
Query end	0.000002
Commit	0.000004
Query end	0.000002
closing tables	0.000003
Unlocking tables	0.000002
closing tables	0.000004
Query end	0.000003
Starting cleanup	0.000002
Freeing items	0.000004
Updating status	0.000021
Reset for next command	0.000003

Ο χρόνος εκτέλεσης αυτής της στρατηγικής είναι 0.00025700 δευτερόλεπτα, που είναι περίπου 38% ταχύτερος από το αρχικό ερώτημα.

Η στρατηγική Hash Join είναι συχνά αποδοτική για μεγάλα σύνολα δεδομένων όπου και οι δύο πίνακες είναι μεγάλοι. Στην περίπτωση μας, παρά την οδηγία IGNORE INDEX, ο βελτιστοποιητής επιλέγει να χρησιμοποιήσει τα ευρετήρια performer_id_idx και performance_id_idx, που οδηγεί σε βέλτιστη απόδοση.

iii. Στρατηγική Merge Join

Η στρατηγική Merge Join υλοποιήθηκε με το ακόλουθο SQL ερώτημα:

```

SELECT
    p.stage_name AS 'Performer',
    ROUND(AVG(r.artist_interpretation), 2) AS 'Avg Artist Interpretation',
    ROUND(AVG(r.overall_impression), 2) AS 'Avg Overall Impression'
FROM
    PERFORMER p FORCE INDEX (idx_performer_stage_name)
JOIN
    PERFORMANCE perf FORCE INDEX (idx_performance_performer_id) ON p.performer_id = perf.performer_id
JOIN
    RATING r FORCE INDEX (idx_rating_performance_id) ON perf.performance_id = r.performance_id
WHERE
    p.stage_name = @artist_name
GROUP BY
    p.performer_id;

```

Το ερώτημα χρησιμοποιεί:

- FORCE INDEX (idx_performer_stage_name): Αναγκάζει τον βελτιστοποιητή να χρησιμοποιήσει το συγκεκριμένο ευρετήριο για τον πίνακα PERFORMER.
- FORCE INDEX (idx_performance_performer_id): Αναγκάζει τον βελτιστοποιητή να χρησιμοποιήσει το συγκεκριμένο ευρετήριο για τον πίνακα PERFORMANCE.
- FORCE INDEX (idx_rating_performance_id): Αναγκάζει τον βελτιστοποιητή να χρησιμοποιήσει το συγκεκριμένο ευρετήριο για τον πίνακα RATING.

Πλάνο Εκτέλεσης Merge Join:

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	p	ref	idx_performer_stage_name	idx_performer_stage_name	137	const	1	Using where; Using index
1	SIMPLE	perf	ref	idx_performance_performer_id	idx_performance_performer_id	4	festival.p.performer_id	3	Using index
1	SIMPLE	r	ref	idx_rating_performance_id	idx_rating_performance_id	4	festival.perf.performance_id	1	

Από το πλάνο εκτέλεσης, παρατηρούμε ότι:

- Για τον πίνακα PERFORMER (p): Χρησιμοποιεί το ευρετήριο idx_performer_stage_name.
- Για τον πίνακα PERFORMANCE (perf): Χρησιμοποιεί το ευρετήριο idx_performance_performer_id.
- Για τον πίνακα RATING (r): Χρησιμοποιεί το ευρετήριο idx_rating_performance_id.

Αποτελέσματα Profiling για Merge Join:

Status	Duration
Starting	0.000047
checking permissions	0.000003
Opening tables	0.000008
After opening tables	0.000003
System lock	0.000003
table lock	0.000003
init	0.000017
Optimizing	0.000014
Statistics	0.000059
Preparing	0.000022
Sorting result	0.000007
Executing	0.000002
Sending data	0.000036
End of update loop	0.000004
Query end	0.000003
Commit	0.000003
Query end	0.000002
closing tables	0.000002
Unlocking tables	0.000003
closing tables	0.000003
Query end	0.000003
Starting cleanup	0.000003
Freeing items	0.000004
Updating status	0.000020
Reset for next command	0.000003

Ο χρόνος εκτέλεσης αυτής της στρατηγικής είναι 0.00027700 δευτερόλεπτα, που είναι περίπου 33% ταχύτερος από το αρχικό ερώτημα.

Η στρατηγική Merge Join λειτουργεί καλύτερα όταν και οι δύο πίνακες είναι ταξινομημένοι στις στήλες join. Στην περίπτωση μας, τα ευρετήρια idx_performer_stage_name, idx_performance_performer_id και idx_rating_performance_id παρέχουν την απαραίτητη ταξινόμηση, επιτρέποντας μια αποδοτική εκτέλεση του ερωτήματος.

Συγκρίνοντας τους χρόνους εκτέλεσης των διαφορετικών στρατηγικών:

- Default Join: 0.00041400 sec
- Nested Loop Join: 0.00037700 sec (9% ταχύτερο από το default)
- Hash Join (BNL): 0.00025700 sec (38% ταχύτερο από το default)
- Merge Join: 0.00027700 sec (33% ταχύτερο από το default)

Η στρατηγική Hash Join (BNL) είναι η πιο αποδοτική για το ερώτημα 4, ακολουθούμενη από τη στρατηγική Merge Join και τη στρατηγική Nested Loop Join.

Πίνακας Σύγκρισης Στρατηγικών:

Strategy	Description	Advantage	Disadvantage
Default Join	Optimizer chooses best strategy	Baseline for comparison	May not always choose optimal plan
Nested Loop Join	Good for small tables with indexes	Efficient when filtering reduces rows significantly	Poor performance on large tables without indexes
Hash Join (BNL)	Good for large unindexed tables	Works well when indexes are missing	Memory intensive for large datasets
Merge Join	Good when data is already sorted	Efficient for large sorted datasets	Requires sorted input or indexes

4.2 Ερώτημα 6

Το ερώτημα 6 αφορά την εύρεση των παραστάσεων που έχει παρακολουθήσει ένας συγκεκριμένος επισκέπτης και τον υπολογισμό του μέσου όρου των αξιολογήσεων για κάθε παράσταση.

Στην παρούσα ανάλυση, θα εξετάσουμε διαφορετικές στρατηγικές βελτιστοποίησης για την εκτέλεση αυτού του ερωτήματος, με στόχο την επίτευξη της βέλτιστης απόδοσης.

4.2.1 Αρχικό Ερώτημα

Το αρχικό SQL ερώτημα που χρησιμοποιήθηκε για την εύρεση των παραστάσεων που έχει παρακολουθήσει ένας επισκέπτης είναι το εξής:

```
SELECT
    perf.performance_id AS 'Performance ID',
    p.stage_name AS 'Performer',
    ROUND(AVG(r.overall_impression), 2) AS 'Avg Overall Impression'
FROM
    RATING r
JOIN
    PERFORMANCE perf ON r.performance_id = perf.performance_id
JOIN
    PERFORMER p ON perf.performer_id = p.performer_id
JOIN
    VISITOR v ON r.visitor_id = v.visitor_id
WHERE
    v.email = @visitor_email
GROUP BY
    r.performance_id, p.stage_name;
```

Πλάνο Εκτέλεσης του Αρχικού Ερωτήματος (Default Join Strategy):

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SINGLE	v	const	PRIMARY,email_UNIQUE,idx_visitor_email				1	
1	SINGLE	r	ref	unique_visitor_performance,visitor_id_idx,performance_id_idx,idx_rating_performance_id,idx_rating_visitor_id	email_UNIQUE	767	const	2	Using index
1	SINGLE	perf	eq_ref	PRIMARY,performer_id_idx,idx_performance_performer_id	unique_visitor_performance	4	const	2	Using where
1	SINGLE	p	eq_ref	PRIMARY	PRIMARY	4	festival.r.performance_id	1	

Από το πλάνο εκτέλεσης, παρατηρούμε ότι ο βελτιστοποιητής επιλέγει:

- Για τον πίνακα VISITOR (v): Χρησιμοποιεί το ευρετήριο email_UNIQUE για γρήγορη αναζήτηση με βάση το email του επισκέπτη.
- Για τον πίνακα RATING (r): Χρησιμοποιεί το ευρετήριο unique_visitor_performance για να βρει τις αξιολογήσεις του συγκεκριμένου επισκέπτη.
- Για τον πίνακα PERFORMANCE (perf): Χρησιμοποιεί το πρωτεύον ευρετήριο PRIMARY για να βρει τις παραστάσεις με βάση το performance_id.
- Για τον πίνακα PERFORMER (p): Χρησιμοποιεί το πρωτεύον ευρετήριο PRIMARY για να βρει τους καλλιτέχνες με βάση το performer_id.

Ο χρόνος εκτέλεσης αυτού του ερωτήματος είναι 0.00032500 δευτερόλεπτα.

4.2.2 Ανάλυση Διαφορετικών Στρατηγικών Join

ι. Στρατηγική Nested Loop Join

Η στρατηγική Nested Loop Join υλοποιήθηκε με το ακόλουθο SQL ερώτημα:


```

SELECT STRAIGHT_JOIN
    perf.performance_id AS 'Performance ID',
    p.stage_name AS 'Performer',
    ROUND(AVG(r.overall_impression), 2) AS 'Avg Overall Impression'
FROM
    VISITOR v
JOIN
    RATING r USE INDEX (PRIMARY) ON v.visitor_id = r.visitor_id
JOIN
    PERFORMANCE perf USE INDEX (PRIMARY) ON r.performance_id = perf.performance_id
JOIN
    PERFORMER p USE INDEX (PRIMARY) ON perf.performer_id = p.performer_id
WHERE
    v.email = @visitor_email
GROUP BY
    r.performance_id, p.stage_name;

```

Το ερώτημα χρησιμοποιεί:

- STRAIGHT_JOIN: Αναγκάζει τον βελτιστοποιητή να εκτελέσει τα joins με τη σειρά που καθορίζονται στο ερώτημα.
- USE INDEX (PRIMARY): Προτείνει στον βελτιστοποιητή να χρησιμοποιήσει το πρωτεύον ευρετήριο των πινάκων RATING, PERFORMANCE και PERFORMER.

Πλάνο Εκτέλεσης Nested Loop Join:

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	v	const	PRIMARY,email_UNIQUE,idx_visitor_email	email_UNIQUE	767	const	1	Using index; Using temporary; Using filesort
1	SIMPLE	r	ALL	NULL	NULL	NULL	NULL	80	Using where
1	SIMPLE	perf	eq_ref	PRIMARY	PRIMARY	4	festival.r.performance_id	1	
1	SIMPLE	p	eq_ref	PRIMARY	PRIMARY	4	festival.perf.performer_id	1	

Από το πλάνο εκτέλεσης, παρατηρούμε ότι:

- Για τον πίνακα VISITOR (v): Χρησιμοποιεί το ευρετήριο email_UNIQUE.
- Για τον πίνακα RATING (r): Δεν χρησιμοποιεί κανένα ευρετήριο (type: ALL), παρά την οδηγία USE INDEX (PRIMARY).
- Για τον πίνακα PERFORMANCE (perf): Χρησιμοποιεί το πρωτεύον ευρετήριο PRIMARY.
- Για τον πίνακα PERFORMER (p): Χρησιμοποιεί το πρωτεύον ευρετήριο PRIMARY.

Επίσης, παρατηρούμε ότι χρησιμοποιείται προσωρινός πίνακας και ταξινόμηση (Using temporary; Using filesort), που μπορεί να επηρεάσει αρνητικά την απόδοση.

Αποτελέσματα Profiling για Nested Loop Join:

Status	Duration
Starting	0.000044
checking permissions	0.000003
Opening tables	0.000012
After opening tables	0.000003
System lock	0.000002
table lock	0.000009
init	0.000022
Optimizing	0.000020
Statistics	0.000042
Preparing	0.000003
Unlocking tables	0.000002
Preparing	0.000029
Creating tmp table	0.001057
Sorting result	0.000014
Executing	0.000019
Query end	0.000004
Removing tmp table	0.000004
Query end	0.000003
Commit	0.000004
Query end	0.000002
closing tables	0.000002
Unlocking tables	0.000003
closing tables	0.000003
Query end	0.000004
Starting cleanup	0.000002
Freeing items	0.000009
Updating status	0.000028
Reset for next command	0.000003

Ο συνολικός χρόνος εκτέλεσης αυτής της στρατηγικής είναι 0.00048400 δευτερόλεπτα, που είναι περίπου 49% βραδύτερος από το αρχικό ερώτημα.

Η στρατηγική Nested Loop Join δεν λειτουργεί καλά σε αυτή την περίπτωση, κυρίως επειδή ο βελτιστοποιητής αγνοεί την οδηγία USE INDEX (PRIMARY) για τον πίνακα RATING και εκτελεί πλήρη σάρωση του πίνακα. Επιπλέον, η χρήση προσωρινού πίνακα και ταξινόμησης αυξάνει τον χρόνο εκτέλεσης.

ii. Στρατηγική Hash Join (BNL)

Η στρατηγική Hash Join υλοποιήθηκε με το ακόλουθο SQL ερώτημα:

```
SELECT
  perf.performance_id AS 'Performance ID',
  p.stage_name AS 'Performer',
  ROUND(AVG(r.overall_impression), 2) AS 'Avg Overall Impression'
FROM
  VISITOR v
JOIN
  RATING r IGNORE INDEX (PRIMARY, idx_rating_visitor_id, idx_rating_performance_id) ON v.visitor_id = r.visitor_id
JOIN
  PERFORMANCE perf IGNORE INDEX (PRIMARY, idx_performance_performer_id) ON r.performance_id = perf.performance_id
JOIN
  PERFORMER p IGNORE INDEX (PRIMARY) ON perf.performer_id = p.performer_id
WHERE
  v.email = @visitor_email
GROUP BY
  r.performance_id, p.stage_name;
```

Το ερώτημα χρησιμοποιεί:

- IGNORE INDEX: Αποτρέπει τον βελτιστοποιητή από το να χρησιμοποιήσει τα συγκεκριμένα ευρετήρια, αναγκάζοντάς τον να χρησιμοποιήσει μια στρατηγική Hash Join.

Πλάνο Εκτέλεσης Hash Join:

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	v	const	PRIMARY,email_UNIQUE,idx_visitor_email	email_UNIQUE	767	const	1	Using index; Using temporary; Using filesort
1	SIMPLE	perf	index	performer_id_idx	performer_id_idx	4	NULL	175	Using index
1	SIMPLE	r	range	unique_visitor_performance,visitor_id_idx,performance_id_idx	unique_visitor_performance	4	NULL	2	Using index condition; Using where; Using join buffer (flat, BNL join)
1	SIMPLE	p	index	NULL	stage_name_UNIQUE	137	NULL	70	Using where; Using index; Using join buffer (incremental, BNL join)

Από το πλάνο εκτέλεσης, παρατηρούμε ότι:

- Για τον πίνακα VISITOR (v): Χρησιμοποιεί το ευρετήριο email_UNIQUE.
- Για τον πίνακα PERFORMANCE (perf): Χρησιμοποιεί το ευρετήριο performer_id_idx, παρά την οδηγία IGNORE INDEX.
- Για τον πίνακα RATING (r): Χρησιμοποιεί το ευρετήριο unique_visitor_performance με τύπο πρόσβασης range.
- Για τον πίνακα PERFORMER (p): Χρησιμοποιεί το ευρετήριο stage_name_UNIQUE.

Επίσης, παρατηρούμε τη χρήση join buffer (flat, BNL join) και (incremental, BNL join), που υποδεικνύει τη χρήση της στρατηγικής Block Nested Loop Join.

Αποτελέσματα Profiling για Hash Join:

Status	Duration
Starting	0.000050
checking permissions	0.000003
Opening tables	0.000026
After opening tables	0.000003
System lock	0.000002
table lock	0.000003
init	0.000018
Optimizing	0.000014
Statistics	0.000055
Preparing	0.000004
Unlocking tables	0.000002
Preparing	0.000028
Creating tmp table	0.000014
Sorting result	0.000023
Executing	0.000002
Sending data	0.000841
Creating sort index	0.000037
End of update loop	0.000007
Removing tmp table	0.000006
End of update loop	0.000004
Query end	0.000004
Commit	0.000004
Query end	0.000003
closing tables	0.000004
Unlocking tables	0.000004
closing tables	0.000005
Query end	0.000014
Starting cleanup	0.000004
Freeing items	0.000008
Updating status	0.000032
Reset for next command	0.000005

Ο συνολικός χρόνος εκτέλεσης αυτής της στρατηγικής είναι 0.00122900 δευτερόλεπτα, που είναι περίπου 278% βραδύτερος από το αρχικό ερώτημα.

Η στρατηγική Hash Join δεν λειτουργεί καλά σε αυτή την περίπτωση, κυρίως επειδή απαιτεί περισσότερη επεξεργασία και μνήμη για τη δημιουργία των hash tables. Επιπλέον, η χρήση προσωρινού πίνακα και ταξινόμησης αυξάνει τον χρόνο εκτέλεσης.

iii. Στρατηγική Merge Join

Η στρατηγική Merge Join υλοποιήθηκε με το ακόλουθο SQL ερώτημα:

```
SELECT
    perf.performance_id AS 'Performance ID',
    p.stage_name AS 'Performer',
    ROUND(AVG(r.overall_impression), 2) AS 'Avg Overall Impression'
FROM
    VISITOR v FORCE INDEX (idx_visitor_email)
JOIN
    RATING r FORCE INDEX (idx_rating_visitor_id) ON v.visitor_id = r.visitor_id
JOIN
    PERFORMANCE perf FORCE INDEX (PRIMARY) ON r.performance_id = perf.performance_id
JOIN
    PERFORMER p FORCE INDEX (PRIMARY) ON perf.performer_id = p.performer_id
WHERE
    v.email = @visitor_email
GROUP BY
    r.performance_id, p.stage_name;
```

Το ερώτημα χρησιμοποιεί:

- FORCE INDEX: Αναγκάζει τον βελτιστοποιητή να χρησιμοποιήσει τα συγκεκριμένα ευρετήρια για κάθε πίνακα.

Πλάνο Εκτέλεσης Merge Join:

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	v	ref	idx_visitor_email	idx_visitor_email	767	const	1	Using where; Using index; Using temporary; Using filesort
1	SIMPLE	r	ref	idx_rating_visitor_id	idx_rating_visitor_id	4	festival.v.visitor_id	1	
1	SIMPLE	perf	eq_ref	PRIMARY	PRIMARY	4	festival.r.performance_id	1	
1	SIMPLE	p	eq_ref	PRIMARY	PRIMARY	4	festival.perf.performer_id	1	

Από το πλάνο εκτέλεσης, παρατηρούμε ότι:

- Για τον πίνακα VISITOR (v): Χρησιμοποιεί το ευρετήριο idx_visitor_email.
- Για τον πίνακα RATING (r): Χρησιμοποιεί το ευρετήριο idx_rating_visitor_id.
- Για τον πίνακα PERFORMANCE (perf): Χρησιμοποιεί το πρωτεύον ευρετήριο PRIMARY.
- Για τον πίνακα PERFORMER (p): Χρησιμοποιεί το πρωτεύον ευρετήριο PRIMARY.

Επίσης, παρατηρούμε τη χρήση προσωρινού πίνακα και ταξινόμησης (Using temporary; Using filesort), που μπορεί να επηρεάσει αρνητικά την απόδοση.

Αποτελέσματα Profiling για Merge Join:

Status	Duration
Starting	0.000010
checking permissions	0.000002
Opening tables	0.000002
After opening tables	0.000002
init	0.000004
Optimizing	0.000004
Executing	0.000004
End of update loop	0.000002
Query end	0.000001
closing tables	0.000002
Query end	0.000002
Starting cleanup	0.000002
Freeing items	0.000002
Updating status	0.000014
Reset for next command	0.000002

Ο συνολικός χρόνος εκτέλεσης αυτής της στρατηγικής είναι 0.00028100 δευτερόλεπτα, που είναι περίπου 14% ταχύτερος από το αρχικό ερώτημα.

Η στρατηγική Merge Join λειτουργεί καλύτερα σε αυτή την περίπτωση, κυρίως επειδή όλοι οι πίνακες έχουν κατάλληλα ευρετήρια που επιτρέπουν την αποδοτική συγχώνευση των ταξινομημένων αποτελεσμάτων.

Συγκρίνοντας τους χρόνους εκτέλεσης των διαφορετικών στρατηγικών:

- Default Join: 0.00032500 sec
- Nested Loop Join: 0.00048400 sec (49% βραδύτερο από το default)
- Hash Join (BNL): 0.00122900 sec (278% βραδύτερο από το default)
- Merge Join: 0.00028100 sec (14% ταχύτερο από το default)

Η στρατηγική Merge Join είναι η πιο αποδοτική για το ερώτημα 6, ακολουθούμενη από την προεπιλεγμένη στρατηγική (Default Join). Οι στρατηγικές Nested Loop Join και Hash Join (BNL) είναι σημαντικά πιο αργές για αυτό το συγκεκριμένο ερώτημα.

Πίνακας Σύγκρισης Στρατηγικών:

Strategy	Description	Advantage	Disadvantage
Default Join	Optimizer chooses best strategy	Baseline for comparison	May not always choose optimal plan
Nested Loop Join	Good for small tables with indexes	Efficient when filtering reduces rows significantly	Poor performance on large tables without indexes
Hash Join (BNL)	Good for large unindexed tables	Works well when indexes are missing	Memory intensive for large datasets
Merge Join	Good when data is already sorted	Efficient for large sorted datasets	Requires sorted input or indexes

5. Για την ανάπτυξη του project, χρησιμοποιήθηκαν:

- MySQL Workbench 8.0.41.CE
- MariaDB 11.7.2
- SQL

6. Git Repository link

Ο σύνδεσμος για το Git Repository είναι ο εξής:

<https://github.com/eleninspl/NTUA-Databases>